PFP: Task-bound claims and the scope of hallucination control

Jeremy H. Carroll

2026-10-05 (v0.1.0)

Abstract

A certificate establishes a declared property of a declared task. It need not establish surrounding prose. We give a finite claim-scoping rule that admits only the claims actually discharged by independent checks, and a counterexample to promoting a correct witness into a correct explanation. This supplies a precise PFP response to some hallucinations while retaining the boundary around unchecked natural-language claims and source quotations.

Mechanism inventory

Slot	Value / Description
Primitive Differences	Distinct candidate answers / witnesses $w, w' \in D$ and predicate value discrepancies $R_j(P, w) \neq R_j(P, w')$
Carriers / Structure	Finite task tuple $P = (D, R, H)$ with finite candidate domain D, solution predicate R, and declared assumptions H
Cochain complex	Finite claim indicator complex $C^0 \xrightarrow{d_0} C^1$ indexing claim obligations $j \in J$ and evaluation receipts
Coboundary maps	Verification indicator map assigning evaluation outcomes $\{0, 1\}$ or error residuals across declared claim indices J
ZeroRel	Exact satisfaction of all declared task predicates $R_j(P, w) = \text{true}$ for $j \in J_{\text{checked}}$
Licensed repairs	Finite candidate substitution $w \mapsto w'$ or prompt re-specification under declared prompt protocol
Quotient / Q1	Quotient of accepted candidate claims modulo unchecked surrounding prose or unsupported universal generalizations
Residuals	Unchecked prose assertions, invented citations, or unverified universal claims outside J_{checked}
Witnesses	Submitted candidate witness $w \in D$ accompanied by typed receipt certifying the admitted subset J_{checked}
Observers / profiles	Independent checker functions $C_j(P, w) \in \{\text{accept}, \text{reject}\}$ evaluating declared predicates
Anomalons	`OPEN`: Semantic drift between natural language explanations and formal predicate specifications
CompletionLaw	Finite checker evaluation exhaustion over the declared claim index set J
Terminal / route	Receipt emission with exact admitted scope J_{checked}, or rejection/refusal on failed required claims
AuthorityEffect	`None`. Receipts certify only that declared checkers accepted specific properties on the submitted witness; they confer zero operational authority

1. Task, claim, and evidence

A task is a tuple $P = (D, R, H)$: a finite candidate domain, a solution predicate, and supplied assumptions. Its identity includes all three. A claim index J names predicates $R_j(P, w)$ with separately supplied checkers C_j. A receipt contains the original task identity, submitted witness, checker identities, and exactly the set J_{checked} for which those checks accepted. An admission schema checks this binding; it does not decide every predicate. Authorization to use the result remains an independent input.

2. Scope theorem

Theorem. Assume each invoked checker is sound on its declared input: $C_j(P, w) = \mathsf{accept} \Rightarrow R_j(P, w)$. A receipt admitting only J_{checked} establishes $\bigwedge_{j \in J_{\text{checked}}} R_j(P, w)$. It establishes no supplied predicate for a different task, assumption set, witness, or uninvoked claim index without an additional checked implication.

Proof. Apply the corresponding soundness implication to each checked index and form their finite conjunction. No implication to another index is among the premises. A countermodel can keep every checked predicate true and make an unchecked predicate false, disproving that further inference. This last argument is a limitation of these premises, not a claim that every pair of mathematical predicates is independent. An actual proved implication can of course transport a claim when its assumptions and task bindings are checked.

3. A correct witness with a false surrounding claim

Let $D = \{0, 1\}^2$ and $R(x, y)$ mean $x + y = 1$. Both $(0, 1)$ and $(1, 0)$ solve the task. The witness $(1, 0)$ passes the equation checker, but the sentence "every solution has $x = 1$" is false. A receipt for equation satisfaction cannot validate that sentence. The companion independently checks the universal claim against all four assignments and rejects it.

An invented source quotation is a third obligation. Its check needs supplied source bytes, their identity, and a span or other declared matching rule. The equation witness supplies none of those. Missing quotation evidence therefore leaves the quotation unverified even when the equation is solved.

4. Two-level PFP exchange

The shared system prompt describes proposing, checking, repair, and refusal, with a worked example. The user instance supplies P, its permitted witness language, required claims, and response schema. The model returns a candidate; the host chooses the relevant declared checkers and replays that candidate. The packet can contain an explanation, but the receipt records its exact checked scope. A failed required claim prevents full acceptance; an optional unchecked explanation remains explicitly outside the accepted scope.

5. Rejection tests and executable evidence

Run the finite replay. It checks a valid equation witness, an invalid witness, a wrong-task submission, a false universal statement, and a missing source quotation. It never replaces a rejected candidate with a separately computed solution. Its enumeration checks the supplied universal claim; it is not a model's answer. There are no provider calls or performance measurements. The proof is written; no new Lean formalization is claimed.

6. What this contributes and what remains open

The logical conjunction rule is elementary. PFP's contribution here is the operational requirement that claim scope, task identity, and independent checking survive prompt and packet handling. It prevents these particular forms of unsupported promotion under the sound-checker assumptions. It does not make all language true, resolve inconsistent assumptions, or establish the soundness of an arbitrary checker solely on the basis of a checker returning `PASS`. General extraction of prose into faithful mathematical obligations remains a separate research and implementation problem.

The two-level teaching packet includes shared system rules, this typed user instance, a different worked example and a required candidate response schema. It keeps provider system/user roles separate. It is a saved example, not a schema-admission or model-run receipt; the local replay checks the mathematical or protocol obligations stated above.

Sources and relation to the programme

This draft builds on the WDP manuscript, Paper 021's source-and-execution distinctions, and the corrected findings notices. Paper 022 supplies a research design whose live-model and offline-strategy claims require their own source-bound scrutiny. These drafts do not adopt its empirical tables as new measurements. The PFP education hub provides the shared-system/typed-instance protocol. Genesis and MSS-N presentation stages do not license a new mathematical or physical carrier.